parallel merge sort on GPU Lecture 03

Step 3: merge all sorted tiles in parallel

- Recall what we Learned:

  $C = merge(A, B)$; <u>Input sequence A and B sorted</u>

  $a_i \in A$, $b_j \in B$;

  $rank(a_i, C) = i + rank(a_i, B)$

  - where $rank(a_i, B)$ is the count of elements $b_j \in B$ with $b_j < a_i$,
  - $rank(a_i, B)$ can be computed with Binary Search

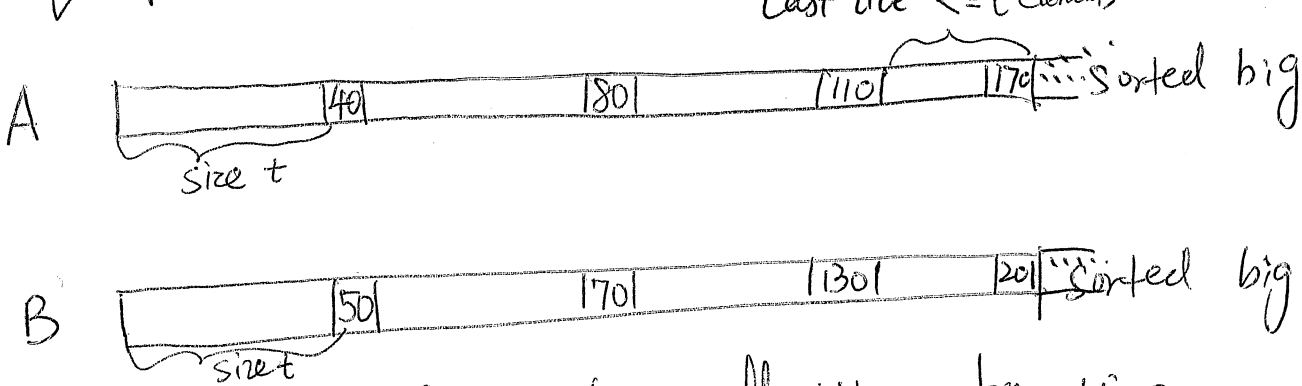
- Homework 4: combines naive tile merge, with each thread block merging two sorted tiles, (no matter How big these tiles are) with Binary Searched Approach.

  <u>As merging proceeds, tiles grows bigger & bigger!</u>

- An elegant, better solution for tile merging!

  ✓ Can we hold the tile size unchanged? by using some tricks?

  ✓ Assume tile size is t, we will not change t.

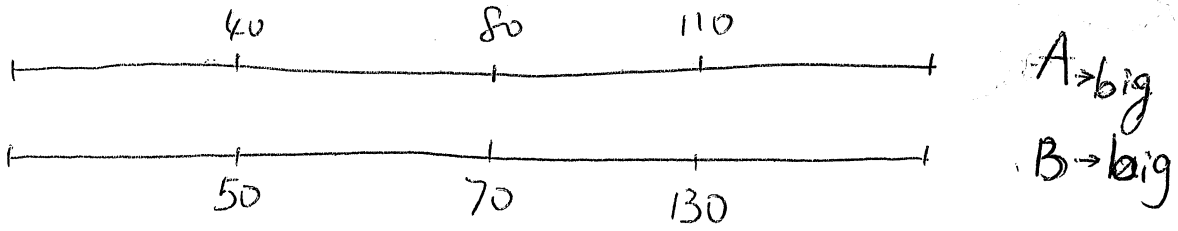


① divide A & B into small tiles, by using every t-th element as <u>splitters</u>.

② we get two splitter sequence: small<=t

$S_A = \{40, 80, 110, 170 \ldots\ldots\}$ Assume

$S_B = \{50, 70, 130, 201 \ldots\ldots\}$ Assume size<=t



use binary-search based merging ⟸ $S = \text{merge}(S_A, S_B) = \{40, 50, 70, 80, 110, 130, 170, 201\}$

we use combined S to split both A & B into contiguous tiles of at most t elements (t = 256 e.g.).

To do so:

① we have to compute the rank of each splitter s in both input sequences; If s = a_i, splitter drawn from A,

$\text{rank}(a_i, A) = i$

$\text{rank}(a_i, B) = ?$ { we first compute

$\text{rank}(s, S_B) = \underbrace{\text{rank}(s, S)}_{\text{known}} - \underbrace{\text{rank}(s, S_A)}_{\text{known}}$

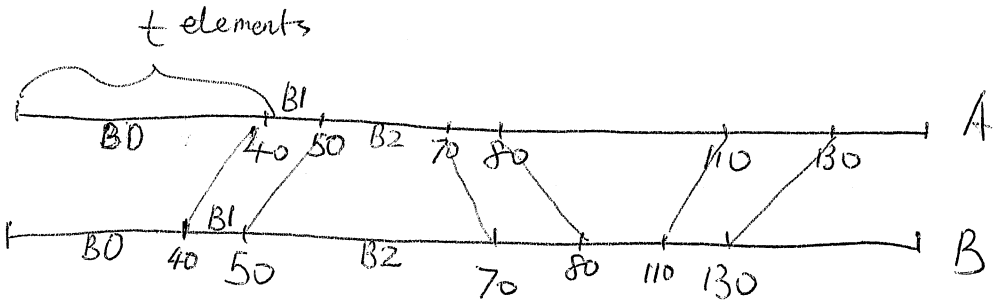
Then we know

$\text{rank}(s, S_B)$ ✓

what does this imply?

⇓

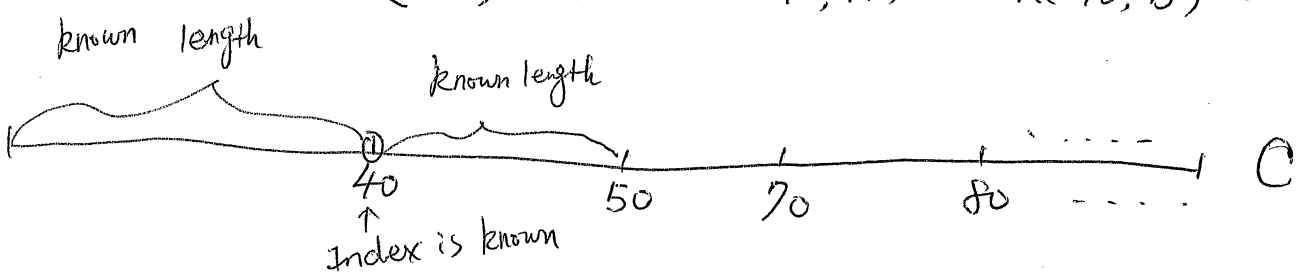
we know a t-element window in B in which s will fall, bounded by the splitters in S_B. Then we determine $\text{rank}(s, B)$ via binary search within that window!



After you get this diagram, GPU comes into and merge them!

Now we know: rank(40, A)
rank(40, B)

Then rank(40, C) = rank(40, A) + rank(40, B)



Then each thread block will merge one small tile T'.

Size(T') <= t (256)